



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SUDESTE DE MINAS GERAIS
Campus Juiz de Fora



Programação I

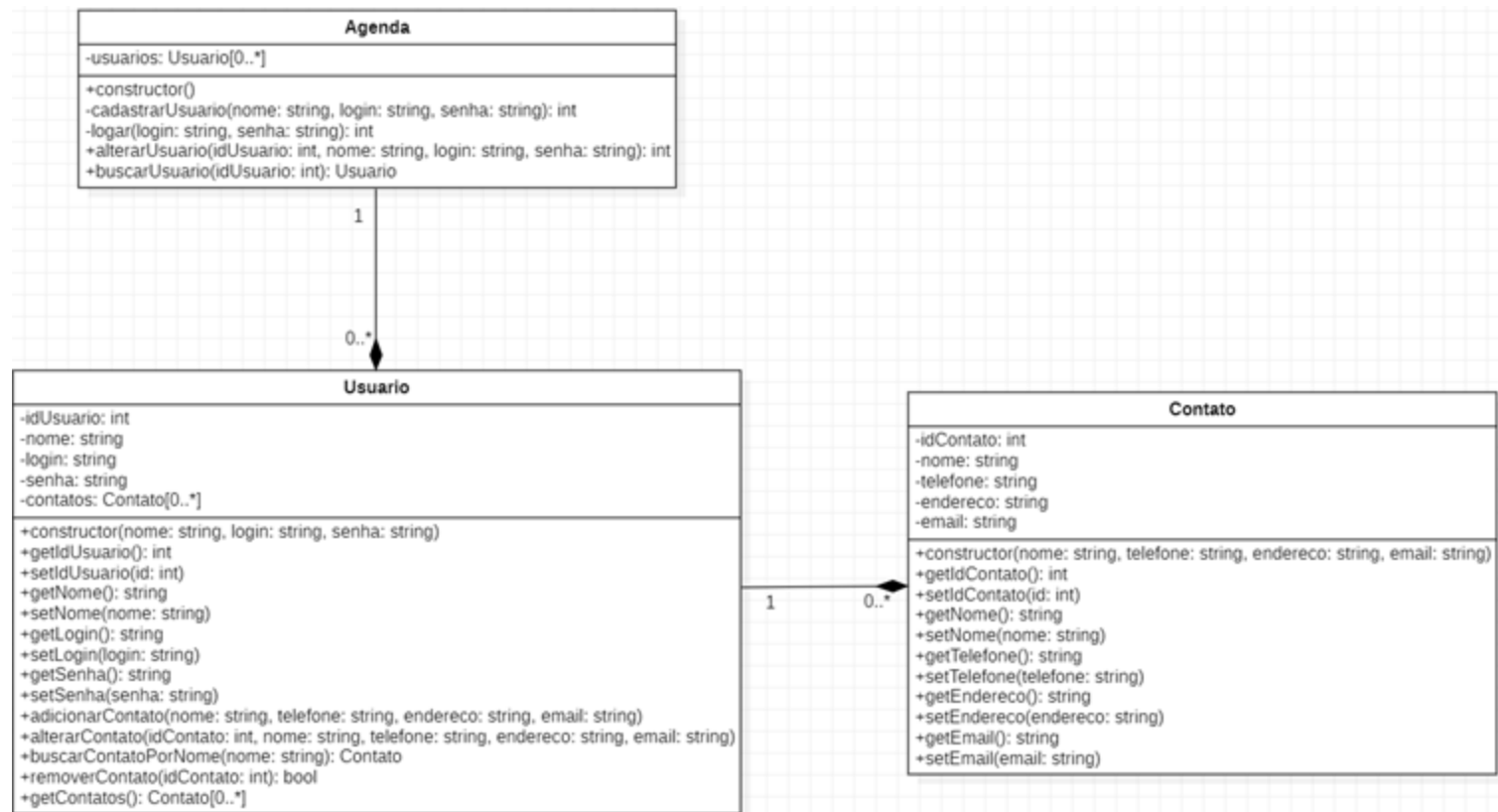
Professor Roberto

roberto.ferreira@ifsudestemg.edu.br

Aula 02 – Sistema Agenda
Codificação das classes

Sistema Agenda

- Diagrama de Classes completo



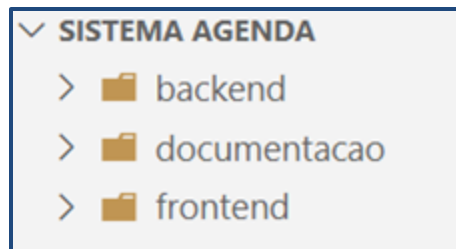
Sistema Agenda

- Vamos criar um projeto utilizando o NodeJS para que possamos criar nosso *backend*, realizar os testes das nossas classes e disponibilizarmos uma API para ser consumida por nosso *frontend*.
- O que é o NodeJS
 - <https://www.youtube.com/watch?v=8VSTrZY8vwl>



Sistema Agenda

- Criar uma pasta denominada Sistema Agenda
 - Esta pasta conterá todo o sistema, composta por duas pastas:
 - frontend:
 - Conterá as páginas web acessíveis aos usuários que serão o ponto de entrada e saída de informação do sistema
 - backend:
 - conterá as classes, as regras de negócio do sistema e proverá uma API para acesso as dados e funcionalidades do sistema



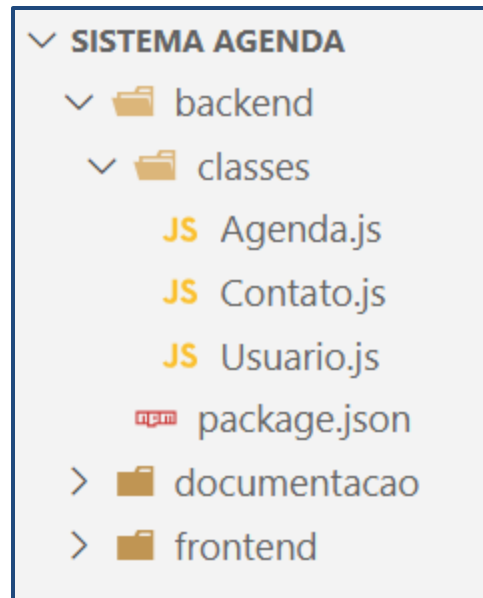
Sistema Agenda

- Criação de um projeto NodeJS para o backend
 - Abrir a pasta Sistema Agenda com o VSCode
 - Abrir o terminal
 - Acessar a pasta backend
 - `cd backend`
 - Criar um projeto node
 - `npm init -y`
 - Abrir o arquivo package.json
 - Incluir `"type": "module"`



Sistema Agenda

- Codificação das classes:
 - Vamos criar uma pasta classes, dentro da pasta backend e vamos criar todos os arquivos que representarão as classes do nosso sistema



Sistema Agenda

- Classe Contato:
 - Representa um contato de um usuário do sistema
 - Possui como atributos
 - idContato
 - nome
 - telefone
 - endereco
 - email

Contato
-idContato: int -nome: string -telefone: string -endereco: string -email: string
+constructor(nome: string, telefone: string, endereco: string, email: string) +getIdContato(): int +setIdContato(id: int) +getNome(): string +setNome(nome: string) +getTelefone(): string +setTelefone(telefone: string) +getEndereco(): string +setEndereco(endereco: string) +getEmail(): string +setEmail(email: string)



Sistema Agenda

- Contato.js

```
export default class Contato {
```

```
  #idContato;
```

```
  #nome;
```

```
  #telefone;
```

```
  #endereco;
```

```
  #email;
```

```
  set nome(nome){
```

```
    const exp = /^[A-Za-z]{3}/;
```

```
    if (nome.match(exp)){
```

```
      this.#nome = nome;
```

```
    } else {
```

```
      throw new Error("O nome deve ter pelo menos 3 caracteres");
```

```
    }
```

```
  }
```

Atributos privados da classe

Expressão regular que representa um texto que contenha no início pelo menos três letras



Sistema Agenda

- Contato.js

```
set telefone(telefone){  
  const exp = /^(\d{2})\s?\d{4,5}-\d{4}$/;  
  /*https://regex101.com/*/  
  if (telefone.match(exp)){  
    this.#telefone = telefone;  
  } else {  
    throw new Error("O telefone deve seguir o formato (99)99999-9999");  
  }  
}  
set endereco(endereco){  
  if (endereco.length >= 10) {  
    this.#endereco = endereco;  
  } else {  
    throw new Error("O endereço deve ter pelo menos 10 caracteres");  
  }  
}
```

Expressão regular para os seguintes formatos de telefone:

(99)9999-9999
(99) 9999-9999
(99)99999-9999
(99) 99999-9999

Testando se o endereço possui pelo menos 10 caracteres



Sistema Agenda

- Contato.js

```
set email(email){  
  const exp = /^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;  
  
  if (email.match(exp)){  
    this.#email = email;  
  } else {  
    throw new Error("O email não corresponde a um endereço válido");  
  }  
}  
  
get nome(){  
  return this.#nome;  
}
```

Expressão regular para testar endereço de e-mail
válido (retirada da internet)



Sistema Agenda

- Contato.js

```
get telefone(){  
    return this.#telefone;  
}  
  
get email(){  
    return this.#email;  
}  
  
get idContato(){  
    return this.#idContato;  
}  
  
get endereco(){  
    return this.#endereco;  
}
```



Sistema Agenda

- Contato.js

```
toString(){  
    return `Nome: ${this.#nome} | Telefone: ${this.#telefone} | Email:  
    ${this.#email} | Endereço: ${this.#endereco}`;  
}
```

método toString() é usado para retornar uma representação textual do objeto

```
constructor(idContato, nome, telefone, endereco, email) {  
    this.nome = nome;  
    this.telefone = telefone;  
    this.endereco = endereco;  
    this.email = email;  
    this.#idContato = idContato;  
}
```

Construtor da classe
Utiliza os métodos set para popular os atributos.
Como o atributo idContato não possui um método set, seu valor é inserido direto no atributo



Sistema Agenda

- Classe Usuario:
 - Representa um usuário do sistema
 - Possui como atributos
 - idUsuario
 - nome
 - login
 - senha
 - contatos

Usuario
-idUsuario: int -nome: string -login: string -senha: string -contatos: Contato[0..*]
+constructor(nome: string, login: string, senha: string) +getIdUsuario(): int +setIdUsuario(id: int) +getNome(): string +setNome(nome: string) +getLogin(): string +setLogin(login: string) +getSenha(): string +setSenha(senha: string) +adicionarContato(nome: string, telefone: string, endereco: string, email: string) +alterarContato(idContato: int, nome: string, telefone: string, endereco: string, email: string) +buscarContatoPorNome(nome: string): Contato +removerContato(idContato: int): bool +getContatos(): Contato[0..*]



Sistema Agenda

- Usuario.js

```
import Contato from "../Contato.js";
export default class Usuario{
  #idUsuario;
  #nome;
  #login;
  #senha;
  #contatos;

  set nome(nome){
    const exp = /^[A-Za-z]{3}/;
    if (nome.match(exp)){
      this.#nome = nome;
    } else {
      throw new Error("O nome deve ter pelo menos 3 caracteres");
    }
  }
}
```

Importando a classe Contato

Atributos privados da classe

Expressão regular que representa um texto que contenha no início pelo menos três letras



Sistema Agenda

- Usuario.js

```
set login(login){  
  const exp = /^[A-Za-z0-9]{5,}$/;  
  if (login.match(exp)){  
    this.#login = login;  
  } else {  
    throw new Error("O login deve ter no mínimo 5 caracteres");  
  }  
}  
  
set senha(senha){  
  const exp = /^[A-Za-z0-9]{8,}$/;  
  if (senha.match(exp)){  
    this.#senha = senha;  
  } else {  
    throw new Error("A senha deve ter no mínimo 8 caracteres");  
  }  
}
```

Expressão regular que caracteriza o login
Texto com no mínimo 5 caracteres contendo
apenas letras e números

Expressão regular que caracteriza o login
Texto com no mínimo 8 caracteres contendo
apenas letras e números



Sistema Agenda

- Usuario.js

```
get nome(){  
    return this.#nome;  
}  
  
get login(){  
    return this.#login;  
}  
  
get senha(){  
    return this.#senha;  
}  
  
get idUsuario(){  
    return this.#idUsuario;  
}
```



Sistema Agenda

- Usuario.js

```
/*  
  Recebe os dados de um contato determina qual será o id dele e o adiciona na lista de  
  contatos.  
*/  
adicionarContato(nome, telefone, endereco, email){  
  const idContato = this.#contatos.length;  
  
  const c1 = new Contato(idContato, nome, telefone, endereco, email);  
  this.#contatos.push(c1);  
}
```

Definindo qual será o id do contato. Por definição do projeto o id será igual ao valor do índice do vetor onde ele será armazenado

Criando o contato

Adicionando o contato no vetor de contatos do usuário. (Atributo #contatos)



Sistema Agenda

- Usuario.js

```
/*  
Verifica se o contato existe, utilizando o idContato  
Se o contato existir altera seus dados, caso contrário dispara um erro  
*/  
alterarContato(idContato, nome, telefone, endereco, email){  
    const contato = this.#contatos[idContato];  
    if (contato !== undefined){  
        contato.nome = nome;  
        contato.telefone = telefone;  
        contato.endereco = endereco;  
        contato.email = email;  
    } else {  
        throw new Error('Contato inexistente');  
    }  
}
```

contato recebe o valor armazenado na posição idContato do vetor

Se existe um contato armazenado faz as alterações dos dados

Se não existir informação armazenada dispara um erro



Sistema Agenda

- Usuario.js

```
/*busca o contato que possui o nome exatamente igual ao nome informado*/
```

```
buscarContatoPorNome(nome){
```

```
    const contatos = this.#contatos
```

```
    let contato = null;
```

```
    /*
```

```
    for (let index = 0; index < contatos.length; index++) {
```

```
        contato = contatos[index];
```

```
        if (contato.nome == nome){
```

```
            return contato;
```

```
        }
```

```
    }
```

```
    */
```

```
    contato = contatos.find((contato) => contato.nome == nome)
```

```
    return contato;
```

```
}
```

Implementação da busca usando o comando for
(comentário)

Implementação da busca usando o método find
Documentação do find:
https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array/find



Sistema Agenda

- Usuario.js

```
/*busca todos os contatos que contenham no nome o texto informado */
```

```
buscarContatosPorNome(texto){
```

```
  const contatos = this.#contatos
```

```
  let contato = [];
```

```
  const exp = new RegExp(texto, 'i');
```

```
  contato = contatos.filter((contato) => contato.nome.match(exp))
```

```
  return contato;
```

```
}
```

Criando uma expressão regular com o valor do texto informado. O segundo parâmetro ('i') indica que a expressão não fará distinção entre letras maiúsculas e minúsculas

Implementação da busca usando o método filter

Documentação do filter:

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array/filter



Sistema Agenda

- Usuario.js

```
/*Remove o contato armazenado no indice idContato. Se não existir o contato no  
índice dispara um erro */
```

```
removerContato(idContato){  
    if (this.#contatos[idContato]){  
        delete this.#contatos[idContato]  
    } else {  
        throw new Error('Contato inexistente');  
    }  
}
```

Se existe o contato na posição informada remove o contato armazenado na posição. Caso contrário dispara um erro.



Sistema Agenda

- Usuario.js

```
constructor(idUsuario, nome, login, senha){  
    this.nome = nome;  
    this.login = login;  
    this.senha = senha;  
    this.#contatos = [];  
    this.#idUsuario = idUsuario;  
}  
}
```

Método construtor da classe

Usa os métodos set para colocar os valores nos atributos.

O atributo #contatos é inicializado com um vetor vazio
[]

Como o atributo idUsuario não possui um método set, seu valor é inserido direto no atributo



Sistema Agenda

- Classe Agenda:
 - Representa o sistema agenda
 - Possui como atributos
 - usuarios

Agenda
-usuarios: Usuario[0..*]
+constructor() -cadastrarUsuario(nome: string, login: string, senha: string): int -logar(login: string, senha: string): int +alterarUsuario(idUsuario: int, nome: string, login: string, senha: string): int +buscarUsuario(idUsuario: int): Usuario



Sistema Agenda

- Agenda.js

```
import Usuario from './Usuario.js';
```

```
class Agenda {
```

```
  #usuarios
```

```
  constructor(){
```

```
    this.#usuarios = []
```

```
  }
```

```
  /* verifica se já existe um usuário como login informado */
```

```
  jaExisteLogin(login){
```

```
    const usuarios = this.#usuarios
```

```
    if (usuarios.find((usuario) => usuario.login == login)){
```

```
      return true;
```

```
    }
```

```
    return false;
```

```
  }
```

Construtor da classe

Inicializa o atributo #usuarios com um vetor vazio



Sistema Agenda

- Agenda.js

```
cadastroUsuario(nome, login, senha){  
    //Se existir algum usuario com o login informado dispara um erro  
    if (this.jaExisteLogin(login)){  
        throw new Error('Já existe um usuário com o login informado')  
    }  
  
    //Se não deu erro cadastra o usuário  
    const idUsuario = this.#usuarios.length;  
    const usuario = new Usuario(idUsuario, nome, login, senha);  
    this.#usuarios.push(usuario);  
}
```



Sistema Agenda

- Agenda.js

```
/*Retorna o usuário com o id informado. Se o usuário não existir no vetor vai
retornar undefined*/
buscarUsuario(idUsuario){
    return this.#usuarios[idUsuario];
}
get usuarios(){
    return this.#usuarios;
}
```



Sistema Agenda

- Agenda.js

```
/*Retorna o idUsuario caso exista um usuário com o login e senha informados. Se não
existir dispara um erro de login e senha inválido*/
logar(login, senha){
    const usuarios = this.#usuarios;
    const usuario = usuarios.find((u) => u.login == login && u.senha == senha);
    if (usuario){
        return usuario.idUsuario;
    }
    throw new Error('Login ou senha inválido');
}
```



Sistema Agenda

- Agenda.js

```
alterarUsuario(idUsuario, nome, login, senha){  
    //Se o novo login já é usado por outro usuário  
    if (this.jaExisteLogin(login)){  
        throw new Error('Não é possível mudar o login. O novo login já é utilizado  
por outro usuário');  
    }  
    //Se não existe um usuário como idUsuario informado  
    const usuario = this.buscarUsuario(idUsuario);  
    if (usuario == undefined) {  
        throw new Error('Não existe um usuário com o id informado');  
    }  
    //Se não disparou nenhum erro altera os dados do usuário  
    usuario.nome = nome;  
    usuario.login = login;  
    usuario.senha = senha;  
}
```

